

GENERALIZED ADAPTIVE MARGIN SOFTMAX

GAM-Softmax

Can we make the softmax margin *smart*
instead of a single number?

Ashish Gupta

One RTX 3050 laptop GPU · 23 May – 31 July 2026

Greeting. This talk is about the last layer of a neural-network classifier — the softmax — and about one number inside a published improvement to it. I will tell you what softmax is and why it needs improving, what the published method does, what I proposed instead, what I built, what I ran, and what came out. I will be straight with you about the outcome: the main hypothesis did *not* hold, and I will show you exactly how I know that. One sub-part did work, and that part is the interesting maths.

Where this talk goes

1. **Introduction** — what softmax is, and the flaw in how we train it
2. **Background** — AS-Softmax and its single number δ
3. **Methodology** — turning δ into a function
4. **Experimentation** — what I ran and what I measured
5. **Results** — the numbers, and the verdict

The whole talk in one line

A classifier is normally told “*be 100 % sure, forever*”. A better instruction is “*win by a margin δ , then stop*”. I asked whether that margin should be **one number** or **a function**.

└─Where this talk goes

Five sections. Roughly two minutes each. The single sentence on the right is the thing to hold on to: cross-entropy, the standard training objective, tells the network to be completely certain and never lets it stop. A margin objective tells it to win by a fixed gap and then stop. My question was whether that gap should stay a single constant, or become a function that varies. Everything else is detail.

1. **Introduction** — what softmax is, and the flow in how we train it.
2. **Background** — AS-Softmax and its single number δ
3. **Methodology** — turning δ into a function
4. **Experimentation** — what I ran and what I measured
5. **Results** — the numbers, and the verdict

The whole talk in one line

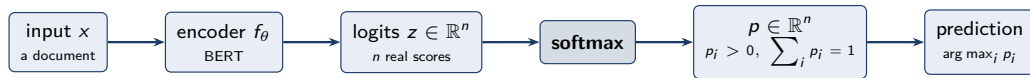
A classifier is normally told "be 100% sure, forever". A better instruction is "win by a margin δ , then stop". I asked whether that margin should be **one number or a function**.

SECTION 1 OF 5

Introduction

Where the softmax sits

The last layer of essentially every classifier



The job it performs

The network emits n unbounded real numbers (“logits”) — they can be negative, they do not sum to anything. Softmax is the map

$$\mathbb{R}^n \longrightarrow \Delta^{n-1}$$

that turns them into a *probability distribution* over the n classes.

Where it is used

- every classifier: text, image, audio
- every language model: the distribution over the next token ($n \approx 50,000$)
- attention weights inside a Transformer
- action probabilities in reinforcement learning

GAM-Softmax

Introduction

Where the softmax sits

Read the pipeline left to right. A document goes into an encoder — in my experiments, BERT — which produces twenty raw scores, one per class. Those scores are arbitrary real numbers. To train with them, and to report a confidence, we need a probability distribution: numbers that are positive and add to one. That conversion is the softmax, and it maps n -dimensional real space onto the probability simplex. It is not a niche component — it is the output layer of every classifier, the next-token distribution of every language model, and the attention weights inside Transformers.

Where the softmax sits

The last layer of essentially every classifier



The job it performs

The network emits n unbounded real numbers ("logits") — they can be negative, they do not sum to anything. Softmax is the map

$$\mathbb{R}^n \rightarrow \Delta^{n-1}$$

that turns them into a probability distribution over the n classes.

Where it is used

- every classifier: text, image, audio
- every language model: the distribution over the next token ($n \approx 50,000$)
- attention weights inside a Transformer
- action probabilities in reinforcement learning

The softmax function

$$p_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}, \quad i = 1, \dots, n$$

Why the formula is built this way

1. **Positive:** $e^{z_i} > 0$ always $\Rightarrow p_i > 0$.
2. **Normalised:** dividing by the sum forces $\sum_i p_i = 1$.
3. **Order-preserving:** e^x is strictly increasing, so $z_i > z_j \Leftrightarrow p_i > p_j$. The prediction is unchanged.
4. **Smooth:** differentiable everywhere $\Rightarrow$ trainable by gradient descent.

Two properties I will use later

- **Shift invariance:** $\text{softmax}(z + c\mathbf{1}) = \text{softmax}(z)$, since $e^{z_i+c} = e^c e^{z_i}$ cancels top and bottom. *Only differences of logits carry information* — which is why a margin is defined on a *difference*.
- **Temperature:** $\text{softmax}(z/T)$ tends to $\arg \max$ as $T \rightarrow 0$ and to uniform as $T \rightarrow \infty$. Softmax is the *smooth* version of $\max$ — hence the name.

The softmax function

$$p_i = \text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}, \quad i = 1, \dots, n$$

Why the formula is built this way

1. **Positives:** $e^x > 0$ always $\Rightarrow p_i > 0$.
2. **Normalized:** dividing by the sum forces $\sum_i p_i = 1$.
3. **Order-preserving:** e^x is strictly increasing, so $x_i > x_j \Leftrightarrow p_i > p_j$. The prediction is unchanged.
4. **Smooth:** differentiable everywhere $\Rightarrow$ trainable by gradient descent.

Two properties I will use later

- **Shift invariance:** $\text{softmax}(x + c\mathbf{1}) = \text{softmax}(x)$, since $e^{x_i+c} = e^c e^{x_i}$ cancels top and bottom. Only differences of logits carry information — which is why a margin is defined on a difference.
- **Temperature:** $\text{softmax}(x/T)$ tends to argmax as $T \rightarrow 0$ and to uniform as $T \rightarrow \infty$. Softmax is the smooth version of max — hence the name.

The formula: exponentiate every score, then divide by the total. Each design choice earns its place. Exponentiation guarantees positivity. Dividing by the sum guarantees the numbers add to one. Because the exponential is strictly increasing, the ordering of the scores is preserved — softmax never changes which class wins. And it is smooth, so we can differentiate through it, which is what makes training possible at all. Two properties matter later. First, shift invariance: adding the same constant to every logit changes nothing, so only *differences* between logits are meaningful — that is exactly why the margin I will show you is defined on a difference of probabilities. Second, with a temperature parameter softmax interpolates between the hard maximum and the uniform distribution: it is a soft max, which is where the name comes from.

A worked example

Four classes, one document

exponentiate $\longrightarrow$ sum $\longrightarrow$ divide

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

class	z_i	e^{z_i}	p_i
sport	3.2	24.53	0.775
politics	1.3	3.67	0.116
medicine	0.2	1.22	0.039
religion	0.8	2.23	0.070
total		31.65	1.000

Check the shift invariance. Subtract 3.2 from every logit:

$$z' = (0, -1.9, -3.0, -2.4)$$

$$e^{z'} = (1, 0.150, 0.050, 0.091), \quad \text{sum} = 1.291$$

$$p' = (0.775, 0.116, 0.039, 0.070) = p$$

Identical. The absolute size of a logit means nothing; only the gaps between logits do. **This is the whole reason a “margin” is the natural thing to control.**

GAM-Softmax

Introduction

A worked example

Concretely, with four classes. Exponentiate: 3.2 becomes 24.5, and note how violently the exponential separates things — a logit gap of 1.9 became a factor of about seven. Sum the exponentials, divide, and you get a valid distribution: 77.5 percent on sport. Now the check on the right. Subtract 3.2 from every logit so the largest becomes zero. The exponentials all change, the sum changes, but every probability is exactly what it was. That is shift invariance, and it is the mathematical reason the rest of this talk is about *differences*: only the gap between the correct class and a competitor carries information.

A worked example

Four classes, one document

exponentiate → sum → divide				$p = \frac{e^{x_i}}{\sum_j e^{x_j}}$
class	x_i	e^{x_i}	p_i	
sport	3.2	24.53	0.775	
politics	1.3	3.67	0.116	
medicine	0.2	1.22	0.039	
religion	0.8	2.31	0.070	
total		31.85	1.000	

Check the shift invariance. Subtract 3.2 from every logit:

$$x' = (0, -1.9, -3.0, -2.4)$$

$$e^{x'} = (1, 0.150, 0.050, 0.091), \text{ sum} = 1.291$$

$$p' = (0.775, 0.116, 0.039, 0.070) = p$$

Identical. The absolute size of a logit means nothing; only the gaps between logits do. This is the whole reason a "margin" is the natural thing to control.

How we train it: cross-entropy

And the flaw hiding in its gradient

With true class t and one-hot label y , the loss is

$$L = - \sum_{i=1}^n y_i \log p_i = -\log p_t \quad (\text{only the true-class term survives})$$

Differentiating through the softmax gives a famously clean gradient:

$$\frac{\partial L}{\partial z_j} = p_j - y_j \quad \implies \quad \frac{\partial L}{\partial z_t} = p_t - 1, \quad \frac{\partial L}{\partial z_j} = p_j \quad (j \neq t)$$

Read the gradient.

- It vanishes *only* when $p_t = 1$ exactly and every $p_j = 0$.
- But $p_i = e^{z_i} / \sum_j e^{z_j} > 0$ strictly — $p_t = 1$ is **unreachable** for finite logits.

Cross-entropy never stops pushing. At $p_t = 0.99$ it still pulls with -0.01 , and it will keep pulling for as long as you train. There is no notion of “this example is done”.

GAM-Softmax

Introduction

How we train it: cross-entropy

Training uses cross-entropy. Because the label is one-hot, the sum collapses to a single term: minus the log of the probability assigned to the correct class. Differentiate that through the softmax and you get one of the tidiest results in machine learning — the gradient with respect to the logits is simply predicted minus actual, p minus y . Now look at when that gradient is zero. It requires the correct class to have probability exactly one. But softmax output is a ratio of positive exponentials, so every probability is strictly positive and exactly one is unreachable. The consequence is the flaw: cross-entropy has no concept of an example being finished. At ninety-nine percent confidence it is still pulling, and it will pull forever.

How we train it: cross-entropy

And the flaw hiding in its gradient

With true class t and one-hot label y , the loss is

$$\mathcal{L} = - \sum_{i=1}^K y_i \log p_i = - \log p_t \quad (\text{only the true-class term survives})$$

Differentiating through the softmax gives a famously clean gradient:

$$\frac{\partial \mathcal{L}}{\partial z_j} = p_j - y_j \quad \Rightarrow \quad \frac{\partial \mathcal{L}}{\partial z_t} = p_t - 1, \quad \frac{\partial \mathcal{L}}{\partial z_j} = p_j \quad (j \neq t)$$

Read the gradient.

- It vanishes only when $p_t = 1$ exactly and every $p_j = 0$.
- But $p_t = e^{z_t} / \sum_i e^{z_i} > 0$ strictly $\rightarrow p_t = 1$ is **unreachable** for finite logits.

Cross-entropy never stops pushing. At $p_t = 0.99$ it still pulls with -0.01 , and it will keep pulling for as long as you train. There is no notion of "this example is done".

Why “never stops” is a real problem

1. **Wasted effort.** With $n = 20$ classes, 19 of the 20 slots per example get gradient at every step — including classes that were hopeless from epoch one.
2. **Overconfidence.** The objective's only fixed point is total certainty, so probabilities get pushed to extremes and stop being honest confidences.
3. **Memorisation of wrong labels.** If a training label is *incorrect*, cross-entropy pushes the model to fit it just as hard. Given enough steps it succeeds.

The point that drives this project

A deep network has enough capacity to fit *arbitrary* labels, including random ones. Cross-entropy gives it no reason to stop — so with noisy labels, it memorises the noise.

The natural fix: **stop pushing once the answer is clearly right**. Now we need to define “clearly”.

Three consequences. First, wasted effort: with twenty classes, nineteen slots per example receive gradient every single step, including classes that were never plausible. Second, overconfidence — the only fixed point of the objective is total certainty, so the probabilities stop being trustworthy confidences. Third, and this is the one that drives the whole project: if a training label is simply wrong, cross-entropy pushes the model to fit the wrong label with exactly the same force. And a large network has enough capacity to succeed — it will memorise the mistake. So the obvious fix is to give the objective a way to say “this example is done, stop”. The rest of the talk is about how to define *done*.

1. **Wasted effort.** With $n = 20$ classes, 19 of the 20 slots per example get gradient at every step — including classes that were hopeless from epoch one.
2. **Overconfidence.** The objective's only fixed point is total certainty, so probabilities get pushed to extremes and stop being honest confidences.
3. **Memorisation of wrong labels.** If a training label is incorrect, cross-entropy pushes the model to fit it just as hard. Given enough steps it succeeds.

The point that drives this project

A deep network has enough capacity to fit arbitrary labels, including random ones. Cross-entropy gives it no reason to stop — so with noisy labels, it memorises the noise.

The natural fix: **stop pushing once the answer is clearly right.** Now we need to define “clearly”.

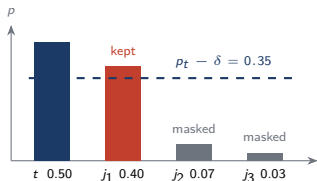
SECTION 2 OF 5

Background

AS-Softmax (Lv et al., 2023) — the method I build on

The rule. For a training example with true class t , *drop* a competing class j from the loss as soon as it is beaten by a margin δ :

$$\text{mask } j \iff p_t - p_j \geq \delta, \quad \mathcal{K}(x) = \{t\} \cup \{j \neq t : p_t - p_j < \delta\}, \quad L_{\text{AS}} = -\log \frac{e^{z_t}}{\sum_{k \in \mathcal{K}(x)} e^{z_k}}$$



With $\delta = 0.15$: j_1 is only 0.10 behind, so it is *still a threat* and stays in the loss. j_2, j_3 are 0.43 and 0.47 behind — beaten by more than δ , so they are dropped and contribute no gradient.

The instruction changes from “*beat everything by everything*” to “*beat everything by δ , then stop*”. Reported: better accuracy, better calibration, $\approx 1.2\times$ faster.

Three details a referee asks about: the test is on *probabilities*, not logits; the mask is *detached*; it is recomputed *every step*, never cached.

GAM-Softmax Background

AS-Softmax (Lv et al., 2023) — the method I build on

AS-Softmax, published in 2023, is the method I set out to extend. Its rule is one line: for a training example, if the correct class already beats a competitor by at least delta in *probability*, delete that competitor from the loss for this step. What remains is ordinary cross-entropy renormalised over the survivors. The picture: the true class is at 0.50, delta is 0.15, so the threshold sits at 0.35. The first competitor at 0.40 is above the line — still a genuine threat — so it stays and keeps producing gradient. The two small ones are far below and get dropped. Three implementation points that a reviewer would ask about: the test is on probabilities, not on raw logits; the mask is detached, so no gradient flows through the threshold itself; and it is recomputed every single step, never cached. The published paper reports better accuracy, better calibration and about a 1.2 times speed-up.

The rule. For a training example with true class t , drop a competing class j from the loss as soon as it is beaten by a margin δ :

$$\text{mask } j \iff p - p_j \geq \delta, \quad K(x) = \{t\} \cup \{j \neq t : p - p_j < \delta\}, \quad \text{Loss} = -\log \frac{e^x}{\sum_{i \in K(x)} e^x}$$



With $\delta = 0.15$, $\hat{p}_t$ is only 0.35 behind, so it is still a threat and stays in the loss. $\hat{p}_j$ are 0.43 and 0.47 behind — beaten by more than δ , so they are dropped and contribute no gradient.

The instruction changes from "beat everything by everything" to "beat everything by δ , then stop". Reported: better accuracy, better calibration, $\approx 1.2\times$ faster.

These details a referee asks about: the test is on probabilities, not logits; the mask is detached; it is recomputed every step, never cached.

The gap in AS-Softmax — and my hypothesis

δ is a single scalar

The same number for “sport vs. politics” and “sport vs. hardware”. The same number for a clean example and a mislabelled one. The same number at step 1 and at step 10,000.

$$\delta \longrightarrow \delta_{t,j}(x, \tau) \quad \tau = \frac{\text{current step}}{\text{total steps}} \in [0, 1]$$

axis	why it might need its own margin
class-pair (t, j)	confusable pairs need a bigger gap than obvious ones
sample x	an ambiguous or mislabelled example is not like a clean one
time τ	early training is noisy; start loose, tighten later

	hypothesis
H0	AS-Softmax beats plain cross-entropy (assumed, never tested)
H1	class-pair structure helps
H2	per-sample structure helps
H3	the axes compose

Success criterion, fixed before running anything: $\geq +0.3\%$ accuracy over AS-Softmax, averaged over seeds. Writing it down in advance is what stops you moving the goalposts afterwards.

GAM-Softmax

Background

The gap in AS-Softmax — and my hypothesis

Here is the gap I went after. Delta is one scalar. It is the same number for a genuinely confusable pair of classes and for two classes that are nothing alike; the same for a clean example and a mislabelled one; the same at the first step and the last. That is a strong assumption and nobody had tested it. My proposal is to replace the scalar by a function of three arguments: the class pair, the sample, and tau — normalised training progress between zero and one. That gives three hypotheses, H1 to H3. Note H0 at the top: the assumption that the method I am extending actually beats plain cross-entropy in the first place. I wrote it down as an assumption, not as something to test. That turns out to be the crux of the whole project. Finally, note the success criterion — plus 0.3 percent over AS-Softmax, averaged over seeds — fixed in advance, before any experiment ran, so I could not move the goalposts afterwards.

The gap in AS-Softmax — and my hypothesis

δ is a single scalar

The same number for "sport vs. politics" and "sport vs. hardware". The same number for a clean example and a mislabelled one. The same number at step 1 and at step 10,000.

$$\delta \rightarrow \delta_{ij}(x, \tau) \quad \tau = \frac{\text{current step}}{\text{total steps}} \in [0, 1]$$

args	why it might need its own margin	hypothesis
class pair (i, j)	confusable pairs need a bigger gap than obvious ones	H0 AS-Softmax beats plain cross-entropy (assumed, never tested)
sample x	an ambiguous or mislabelled example is not like a clean one	H1 class pair structure helps
time τ	early training is risky: start loose, tighten later	H2 per sample structure helps H3 the same sample

Success criterion, fixed before running anything: $\geq +0.3\%$ accuracy over AS-Softmax, averaged over seeds. Writing it down in advance is what stops you moving the goalposts afterwards.

SECTION 3 OF 5

Methodology

GAM-Softmax: the general form

$$\mathcal{K}(x) = \{t\} \cup \{j \neq t : p_t - p_j < \delta_{t,j}(x, \tau)\}, \quad L = -\log \frac{e^{z_t}}{\sum_{k \in \mathcal{K}(x)} e^{z_k}}$$

Same loss as before — the *only* change is that the threshold is now a function rather than a constant.

Reduction property (unit-tested)

If $\delta_{t,j}(x, \tau) \equiv \delta$ is constant, then $\mathcal{K}$ is *exactly* AS-Softmax's kept set, so $L \equiv L_{AS}$ — not approximately, identically.

Two familiar losses fall out

$\delta \geq 1 \Rightarrow$ nothing is ever masked (since $p_t - p_j \leq 1$)
 $\Rightarrow$ **plain cross-entropy**.
 δ constant $\Rightarrow$ **AS-Softmax**.

This matters for honesty, not elegance: because the general form *contains* the baseline, any difference in the results is attributable to the margin function alone — not to a different loss, a different optimiser, or a different bug.

Enforced by a test that asserts equality of the two losses to floating-point tolerance. 70 unit tests, ≈ 3 s.

GAM-Softmax

Methodology

GAM-Softmax: the general form

The generalisation is deliberately minimal. Same masking rule, same renormalised loss — the single change is that the threshold on the right of the inequality is now a function of the class pair, the sample and the time, instead of a constant. Two consequences worth stating. If the function is constant, the kept set is exactly AS-Softmax's kept set and the losses are identical — I assert that with a unit test, so it is checked on every commit. And if delta is at least one, nothing can ever be masked, because a difference of two probabilities can never exceed one — so you recover plain cross-entropy. Both baselines are special cases of my formulation. That matters for scientific honesty rather than elegance: because the general form contains the baseline exactly, any difference in the numbers has to come from the margin function itself.

$$K(x) = \{i \mid \exists j \neq i : p_i - p_j < \delta_{ij}(x, \tau)\}, \quad L = -\log \frac{e^{x_i}}{\sum_{i \in K(x)} e^{x_i}}$$

Same loss as before — the only change is that the threshold is now a function rather than a constant.

Reduction property (unit-tested)

If $\delta_{ij}(x, \tau) = \delta$ is constant, then L is exactly AS-Softmax's kept set, so $L = L_{AS}$ — not approximately, identically.

Two familiar losses fall out

$\delta \geq 1 \Rightarrow$ nothing is ever masked (since $p_i - p_j \leq 1$)
 $\Rightarrow$ plain cross-entropy
 δ constant $\Rightarrow$ AS-Softmax.

This matters for honesty, not elegance: because the general form contains the baseline, any difference in the results is attributable to the margin function alone — not to a different loss, a different optimiser or a different bug.

Enforced by a test that asserts equality of the two losses for floating-point tolerance. 10 such tests, in 3s.

M3 — the class-pair axis

$$\delta_{t,j} = \delta_{\min} + (\delta_{\max}(\tau) - \delta_{\min}) \cdot \sigma\left(\frac{u_t \cdot v_j}{\sqrt{k}}\right), \quad u, v \in \mathbb{R}^{n \times k}, \quad k = 8$$

Reading the formula

- every class c gets two learnable vectors u_c, v_c of length $k = 8$; their inner product scores the *ordered* pair (t, j)
- σ (the logistic sigmoid) squashes that score into $(0, 1)$, so $\delta_{t,j}$ can never leave $[\delta_{\min}, \delta_{\max}]$
- $\delta_{\max}(\tau)$ is the *time* axis: a warm-up schedule
- dividing by $\sqrt{k}$ keeps the pre-sigmoid scale $O(1)$ — the same variance argument as in attention

Why low-rank, not a full table?

A free margin per ordered pair is $n(n-1) = 380$ unconstrained numbers for $n = 20$, with no sharing between classes.

The low-rank form uses $2nk = 320$ and *forces* classes to share structure — which is the actual hypothesis: that pair difficulty is low-dimensional.

$$\delta_{kj} = \delta_{\max} + (\delta_{\max}(\tau) - \delta_{\min}) \cdot \sigma\left(\frac{\delta_{kj} - \delta_{\min}}{\sqrt{k}}\right), \quad u, v \in \mathbb{R}^{2 \times 8}, \quad k \in \mathbb{N}$$

Reading the formula

- every class c gets two learnable vectors u_c, v_c of length $k \in \mathbb{N}$; their inner product scores the ordered pair $\{c, j\}$
- σ (the logistic sigmoid) squashes that score into $(0, 1)$, so δ_{kj} can never leave $[\delta_{\min}, \delta_{\max}]$
- $\delta_{\max}(\tau)$ is the time axis: a warm-up schedule
- dividing by $\sqrt{k}$ keeps the pre-sigmoid scale $\mathcal{O}(1)$ — the same variance argument as in attention

Why low-rank, not a full table?

A free margin per ordered pair is $n(n-1) = 380$ unconstrained numbers for $n = 20$, with no sharing between classes. The low-rank form uses $2nk = 320$ and forces classes to share structure — which is the actual hypothesis: that pair difficulty is low-dimensional.

The first axis. Each class gets two short learnable vectors, u and v , of length eight. The margin for the ordered pair — true class t against competitor j — is the inner product of $u-t$ and $v-j$, pushed through a sigmoid so it is bounded, and then stretched between a floor and a time-varying ceiling. The square-root-of- k divisor is the same variance normalisation used in attention: it stops the inner product from saturating the sigmoid at initialisation. Why low-rank rather than a free table? For twenty classes a free table is 380 unconstrained numbers with no sharing at all, and it would just overfit. The factored form is 320 parameters that force classes to share structure — and that sharing *is* the hypothesis: that how confusable a pair is, is a low-dimensional property.

M4 — the sample axis

Step 1 — how suspicious is this example, relative to its batch?

$$s_i = \sigma\left(\frac{\bar{p}_t - p_{t,i}}{\text{std}(p_t) \cdot \text{temp}}\right) \in (0, 1), \quad s_i > 0.5 \iff \text{less confident than its peers}$$

Step 2 — move the margin by that amount

$$\delta_i(\tau) = \text{clamp}\left(\underbrace{\delta_{\text{base}}(\tau)}_{\text{time axis}} - \beta(\tau)(2s_i - 1), \delta_{\text{floor}}, \delta_{\text{ceil}}\right)$$

- suspicious ($s_i \rightarrow 1$) $\Rightarrow \delta$ down $\Rightarrow$ more classes masked $\Rightarrow$ less gradient
- confident ($s_i \rightarrow 0$) $\Rightarrow \delta$ up, but capped at AS-Softmax's own δ — the axis is **one-sided**, so any difference comes purely from what we do to suspicious samples

Two deliberate design choices

$(2s_i - 1)$ averages to ≈ 0 over a batch, so this varies δ across samples without changing the overall masking level — that stays the time axis's job, keeping the axes separable.

$\beta(\tau) = 0$ for the first 30 % of training: early on p_t is near-random and carries no signal about which labels are wrong.

GAM-Softmax

Methodology

M4 — the sample axis

The second axis, and this is the one that worked. Two steps. First, score how suspicious each example is, using only information the model already computed: take the probability it currently assigns to that example's training label, compare it to the batch mean, divide by the batch standard deviation, and squash with a sigmoid. Above one half means “this example is less confident than its peers” — that is, suspicious. Second, shift the margin down by that amount. Suspicious examples get a lower margin, so more of their competitors get masked, so they contribute less gradient. Two design choices I would defend to a referee. The correction is mean-centred, so it redistributes the margin across samples without changing the average masking level — that keeps this axis independent of the time axis, which matters for the ablation. And beta is zero for the first thirty percent of training, because early on the model's confidence is near-random and tells you nothing about which labels are wrong.

M4 — the sample axis

Step 1 — how suspicious is this example, relative to its batch?

$$s = \sigma\left(\frac{p_i - p_{\text{batch}}}{\text{std}(p_{\text{batch}}) \cdot \text{temp}}\right) \in (0, 1) \quad s > 0.5 \iff \text{less confident than its peers}$$

Step 2 — move the margin by that amount

$$\delta_i(\tau) = \text{clamp}\left(\underbrace{\delta_{\text{base}}(\tau)}_{\text{mean axis}} - \beta(\tau) \cdot (2s_i - 1), \delta_{\text{min}}, \delta_{\text{max}}\right)$$

- suspicious ($s \rightarrow 1$) $\Rightarrow \delta$ down $\Rightarrow$ more classes masked $\Rightarrow$ less gradient
- confident ($s \rightarrow 0$) $\Rightarrow \delta$ up, but capped at AS-Softmax's own δ — the axis is *one-sided*, so any difference comes purely from what we do to suspicious samples

Two deliberate design choices

$(2s_i - 1)$ averages to ≈ 0 over a batch, so this varies δ across samples without changing the overall masking level — that stays the time axis's job, keeping the axes separable.

$\beta(\tau) = 0$ for the first 30% of training: early on p_i is near-random and carries no signal about which labels are wrong.

The idea worth keeping: let δ go negative

Nothing in the mathematics requires a margin to be positive. Since $p_t - p_j \in [-1, 1]$, the value $\delta = -0.85$ is a perfectly legal threshold. It reads:

“drop class j even if it is currently beating the training label by up to 0.85.”

On a mislabelled example

- the model keeps p_t low (the label is wrong)
- so with $\delta > 0$, *no* competitor ever clears the bar
- nothing is masked $\Rightarrow$ full cross-entropy $\Rightarrow$ pushed to fit the wrong label forever
- that is exactly how a network memorises noise

With $\delta < 0$, the algebra ends it

$$\mathcal{K}(x) = \{t\} \Rightarrow p_t^{\text{renorm}} = \frac{e^{z_t}}{e^{z_t}} = 1$$

$$\Rightarrow L = -\log 1 = 0 \Rightarrow \nabla L = 0$$

The suspicious sample *removes itself* from training
— no threshold rule, no sample-selection heuristic.

The reframing

The noisy-label literature’s “small-loss trick” — drop the high-loss examples — is exactly the special case $\delta < 0$ of a *margin*. No second network, no auxiliary loss, no selection schedule: one line of configuration.

GAM-Softmax

Methodology

— The idea worth keeping: let δ go *negative*

This is the slide I would keep if I could keep only one. Nothing in the maths says a margin has to be positive. The quantity being thresholded is a difference of two probabilities, so it lives between minus one and plus one, and minus 0.85 is a legal threshold. It means: drop this competitor even if it is currently *ahead* of the training label. Now follow a mislabelled example. Because the label is wrong, the model keeps assigning it low probability. So under a positive margin no competitor ever clears the bar, nothing gets masked, and the example receives full cross-entropy pressure forever — that is the memorisation mechanism, precisely. Under a negative margin, every competitor is masked, the kept set is the single true class, the renormalised probability is e-to-the-z over e-to-the-z which is exactly one, the log of one is zero, and the gradient is zero. The example removes itself. And that gives the reframing in the box: dropping high-loss samples — a standard trick in the noisy-label literature that usually needs a second network or a selection schedule — is just a margin with a negative delta. One config line.

The idea worth keeping: let δ go negative

Nothing in the mathematics requires a margin to be positive. Since $p - \mu \in [-1, 1]$, the value $\delta := -0.85$ is a perfectly legal threshold. It reads:

"drop class j even if it is currently beating the training label by up to 0.85."

On a mislabelled example

- the model keeps p_i low (the label is wrong)
- so with $\delta > 0$, no competitor ever clears the bar
- nothing is masked $\Rightarrow$ full cross-entropy $\Rightarrow$ pushed to fit the wrong label forever
- this is exactly how a network memorises noise

With $\delta < 0$, the algebra ends it

$$K(x) = \{i\} \Rightarrow \hat{p}_i^{\text{norm}} = \frac{e^{z_i}}{\sum_j e^{z_j}} = 1$$

$$\Rightarrow L = -\log 1 = 0 \Rightarrow \nabla L = 0$$

The suspicious sample removes itself from training — no threshold rule, no sample-selection heuristic.

The reframing

The noisy-label literature's "small-loss trick" — drop the high-loss examples — is exactly the special case $\delta < 0$ of a margin. No second network, no auxiliary loss, no selection schedule: one line of configuration.

SECTION 4 OF 5

Experimentation

The setup

Constraint that shaped everything

One **RTX 3050 laptop, 6 GB** (the plan assumed a 3090/4090). One 20-Newsgroups run ≈ 25 min $\Rightarrow$ a properly seeded sweep is a day of wall clock.

Testbed

- 20 Newsgroups, 20 classes, headers/footers/quotes *stripped* — otherwise BERT reaches $\approx 99\%$ from metadata and no method can separate from another
- BERT-base, 4 epochs, batch 16, lr 2×10^{-5} , seq len 128
- 6 configs, **identical** except the loss: block; 2 seeds (42, 43)

Label noise — and why it is a *paired* comparison

40 % symmetric label noise on the training set; validation labels stay clean.

The noise seed is fixed and *decoupled* from the run seed, so every method sees the *identical* corrupted labels. Methods are compared on the same corrupted dataset, not on statistically similar ones.

One config file = one experiment = one row of the results table. 9 loss functions, 70 unit tests running in ≈ 3 s. The null result is not a code failure.

2026-08-03

GAM-Softmax

Experimentation

The setup

The hardware constraint is real and it shaped every choice: a six-gigabyte laptop GPU, where one run of this dataset takes about twenty-five minutes, so a properly seeded comparison is a full day. The testbed is 20 Newsgroups with headers, footers and quotes stripped — without stripping, BERT scores about ninety-nine percent by reading email metadata and every method ties, so the experiment would measure nothing. Then the key protocol point on the right: I corrupt forty percent of the training labels, at random, to a different class, while validation labels stay clean. The noise seed is fixed and separate from the run seed, which means every method sees the exact same corrupted labels. So this is a paired comparison — the methods differ, the data does not. Finally the engineering discipline: one config file is one experiment is one row of the results table, and seventy unit tests pass in about three seconds. When I show you a null result, it is not a bug.

The setup

Constraint that shaped everything
One RTX 3050 laptop, 6 GB (the plan assumed a 3000/4000). One 20-Newsgroups run $\approx$ 25 min $\Rightarrow$ a properly seeded sweep is a day of wall clock.

Testbed

- 20 Newsgroups, 20 classes, headers/footers/quotes stripped — otherwise BERT reaches $\approx$ 99% from metadata and no method can separate from another

- BERT-base, 4 epochs, batch 16, lr 2×10^{-6} , seq len 128
- 6 configs, identical except the loss: block; 2 seeds (42, 43)

Label noise — and why it is a paired comparison

40% asymmetric label noise on the training set; validation labels stay clean.

The noise seed is fixed and decoupled from the run seed, so every method sees the identical corrupted labels. Methods are compared on the same corrupted dataset, not on statistically similar ones.

One config file $\equiv$ one experiment $\equiv$ one row of the results table. 9 loss functions, 70 unit tests running in $\approx$ 3s. The null result is not a code failure.

What I measure — and the arithmetic behind it

Accuracy alone had been flat in every earlier experiment, so E1 measures the *mechanism* directly.

- **peak** and **final** validation accuracy, and the **decay** between them — the overfitting trajectory
- **memorisation rate:** of the training examples whose labels were *corrupted*, the fraction the model now predicts *as their corrupted label*
- **recovery rate:** same examples, fraction predicted as their *true* label despite being trained on a wrong one
- **masked ratio**, and 15-bin **ECE** (calibration)

The no-memorisation floor

With 40 % noise, where a flip never returns the original label, a model that learned the *true* function and memorised *nothing* scores exactly

$$0.6 \times 100 \% + 0.4 \times 0 \% = 60.0 \%$$

on the labels it was given.

Everything above 60 is memorised noise — a quantity, not an impression.

What I measure — and the arithmetic behind it

Six weeks of reading only accuracy taught me nothing, because accuracy was flat everywhere. So the final experiment measures the mechanism. The central quantity is the memorisation rate: I keep a record of exactly which training examples had their labels corrupted, and at the end I ask what fraction of those the model now predicts as the wrong label it was trained on. That is memorisation measured directly, not inferred from an accuracy curve. And the box on the right is the arithmetic that makes it interpretable. Sixty percent of the training labels are still correct; forty percent were flipped, and a flip never returns the original label. So a model that perfectly learned the true function and memorised nothing would score exactly sixty percent against the labels it was handed. Everything above sixty is memorised noise. That gives a principled zero point.

Accuracy alone had been flat in every earlier experiment, so E1 measures the mechanism directly.

- **peak and final validation accuracy**, and the **decay** between them — the overfitting trajectory
- **memorisation rate**: of the training examples whose labels were corrupted, the fraction the model now predicts as their corrupted label
- **recovery rate**: same examples, fraction predicted as their true label despite being trained on a wrong one
- **masked ratio**, and 15-bin **ECE** (calibration)

The no-memorisation floor

With 40% noise, where a flip never returns the original label, a model that learned the true function and memorised nothing scores exactly

$$0.6 \times 100\% + 0.4 \times 0\% = 60.0\%$$

on the labels it was given.

Everything above 60 is memorised noise — a quantity, not an impression.

Every experiment, in order

#	testbed	question	result
1	SST-5, clean, 3 seeds	baselines sane?	softmax 51.1 %, AS-Softmax 52.2 % $\Rightarrow$ AS $>$ CE by +1.1 $\checkmark$
2	SST-5, clean, 3 seeds	H1, fixed margin	M3 52.10 vs AS 52.17 = -0.06
3	SST-5, clean, 3 seeds	H1, learnable margin	M3' 51.83 vs AS 52.17 = -0.33
4	SST-5, tuning screen	is H1 under-tuned?	higher margin LR hurt <i>monotonically</i> (52.5 $\rightarrow$ 50.6 $\rightarrow$ 50.1) $\Rightarrow$ not a tuning artefact
5	20NG, clean, 1 seed	testbed with head-room?	GAM 71.18 $>$ AS 70.59, but plain CE 71.46 wins $\Rightarrow$ AS $<$ CE
6	SST-5, noise {0, .2, .4}	does a hard regime rescue it?	all gaps inside $\pm 2\%$ seed scatter $\Rightarrow$ inconclusive
7	20NG, 40 % noise, 2 seeds	the decisive test	next section

H0 kept failing. AS-Softmax beat cross-entropy on SST-5 (+1.1), lost on 20NG (-0.9), and was a wash under noise. Three of four setups gave the method I extend *no edge* over the simplest possible baseline.

The one repeatable GAM signal was stability, not accuracy: lowest seed-to-seed variance at every noise level, and still *improving* at the last epoch where the baselines had peaked and turned over.

GAM-Softmax

Experimentation

Every experiment, in order

Every experiment, in order

#	method	question	result
1	SST-5, clean, 1 seed	baselines same?	softmax 51.1%, AS-Softmax 52.2% $\rightarrow$ AS $>$ CE by +2.1%
2	SST-5, clean, 1 seed	H0: fluid margin	AS 52.12 vs AS 52.17 $\rightarrow$ $\sim$ 0.05
3	SST-5, clean, 1 seed	H0: learnable margin	AS 51.83 vs AS 52.17 $\rightarrow$ $\sim$ 0.33
4	SST-5, tuning screen	n H0 under-tuned?	higher margin LR lost monotonically 52.5 $\rightarrow$ 52.8 $\rightarrow$ 60.1 $\rightarrow$ not a tuning artifact
5	20NG, clean, 1 seed	method with local-mins?	GAM 71.18 $>$ AS 70.59, but plain CE: 71.48
6	SST-5, noise [2..2..4]	does a hard regime exist at 2?	noise vs AS $<$ CE, all gaps inside $>$ 2% seed noise $\rightarrow$ intractable
7	20NG, 60% noise, 2 seeds	the decision test	most unstable

H0 kept failing: AS-Softmax beat cross-entropy on SST-5 (+1.1), lost on 20NG (-0.9), and was a wash under noise. Three of four setups gave the method I extend no edge over the simplest possible baseline.

The one repeatable GAM signal was stability, not accuracy: lowest seed-to-seed variance at every noise level, and still improving at the last epoch where the baselines had peaked and turned over.

The honest audit trail — every experiment, in the order I ran them. Row one: baselines reproduce, AS-Softmax beats plain softmax by about one point on SST-5, so the engine is trustworthy. Rows two and three: the class-pair hypothesis, tested twice, comes out slightly negative both times. Row four is the one I would want a referee to see — I checked whether the flat result was just bad tuning, and raising the margin learning rate made it monotonically *worse*, so it is a real effect, not under-tuning. Row five moves to a harder dataset and finds the thing that eventually decided the project: plain cross-entropy wins, and AS-Softmax — the method I am extending — loses to it. Row six, adding label noise, is inconclusive. Two patterns repeat. First, H0 — my unexamined assumption — kept failing. Second, the only repeatable signal from my method was stability rather than accuracy, and that is what the final experiment was designed to confirm or kill.

SECTION 5 OF 5

Results and Demonstration of Final Result

E1 — the decisive experiment

20 Newsgroups, 40 % symmetric label noise, BERT-base, 2 seeds (42, 43)

method	peak val %	final val %	decay	memorised noise (pts over floor)	mem @final %	recovers true %
softmax (plain CE)	67.59 ± 0.71	64.19 ± 0.92	-3.40	+11.5	33.7 ± 0.4	41.2
as_softmax (scalar δ)	67.56 ± 0.34	64.72 ± 0.20	-2.84	+13.6	39.9 ± 1.8	44.1
gam_m3 (class-pair)	67.68 ± 0.81	65.21 ± 1.42	-2.47	+11.4	35.2 ± 1.7	45.9
gam_m4 (sample)	68.09 ± 0.05	66.96 ± 0.42	-1.13	+3.1	19.3 ± 0.7	57.3

Read the accuracy column first — it is flat

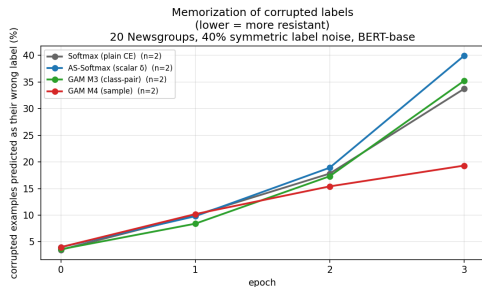
Peak accuracy spans 67.56 to 68.09: half a point across all four methods. Plain cross-entropy alone spans **1.0 point** between its two seeds. **The accuracy column contains no result.**

Now read the memorisation column

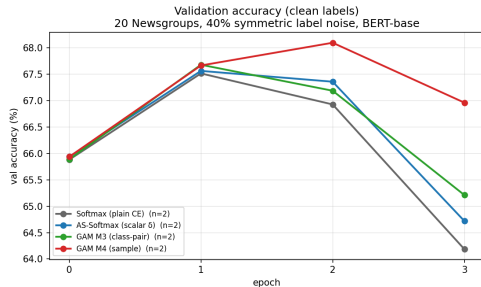
The same two seeds of plain cross-entropy span only **0.6 points** of memorisation. So a **14-point** gap is $\approx 20\times$ the measurement's own noise — it cannot be a lucky seed.

Here is the whole experiment on one slide. Read the accuracy column first, because the discipline matters more than the result: peak accuracy runs from 67.6 to 68.1, about half a point across four quite different objectives. Meanwhile plain cross-entropy on its own spans a full point between its two random seeds. So the accuracy column contains no result — every difference in it is smaller than the noise of the measurement. Now the memorisation column. The same two seeds of cross-entropy span only six tenths of a point there, so that measurement is roughly twenty times quieter. And the gap between the bottom row and the rest is fourteen points. That is about twenty times the noise floor of the measurement, which is why I am willing to make a claim about that column and not about the accuracy column. Choosing which metric can support a claim is most of this experiment.

The output, in two pictures



Lower is better. All methods rise — M4's curve flattens while the rest accelerate.



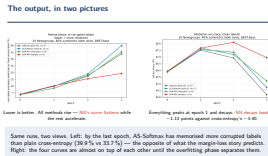
Everything peaks at epoch 1 and decays. M4 decays least —
—1.13 points against cross-entropy's —3.40.

Same runs, two views. Left: by the last epoch, AS-Softmax has memorised *more* corrupted labels than plain cross-entropy (39.9 % vs 33.7 %) — the opposite of what the margin-loss story predicts. Right: the four curves are almost on top of each other until the overfitting phase separates them.

GAM-Softmax

Results and Demonstration of Final Result

The output, in two pictures



The two pictures are the same four runs, viewed two ways. On the left, memorisation of corrupted labels over the four epochs. Everything climbs — all of these models memorise — but the red curve, the sample-axis variant, flattens out around fifteen to nineteen percent while the others accelerate past thirty-five. Note the blue curve, AS-Softmax: it ends *highest*, above plain cross-entropy. I will come back to that, because it contradicts the standard justification for these losses. On the right, validation accuracy on clean labels. Everything peaks at epoch one and then decays as the models start fitting the noise. The red curve decays least — about one point lost against cross-entropy's three and a half. Same mechanism, two views: less memorisation, less decay.

What the numbers actually say

1. The precondition failed.

AS – CE = -0.03 points over two seeds (-0.29 and $+0.23$: a coin flip). *Fourth testbed in a row.* The method I set out to generalise has, in my hands, no advantage over plain cross-entropy to generalise.

2. AS-Softmax memorises *more* than plain CE.

$+13.6$ vs $+11.5$ points over the floor, on *both* seeds ($+7.8$, $+4.6$). This contradicts the standard story — and my own reason for moving to noisy labels.

Mechanism: masking retires the *easy* negatives first — what a correctly-labelled example runs out of early. So an ever-larger share of the gradient goes to the still-confusable examples, which are exactly the mislabelled ones. **Scalar masking concentrates effort onto the noise.**

3. The sample axis does what it was designed to do.

(2-seed means)	CE	AS	M3	M4
memorised noise (pts)	11.5	13.6	11.4	3.1
recovers true label	41.2	44.1	45.9	57.3
peak-to-final decay	3.40	2.84	2.47	1.13
non-target slots masked	0 %	56 %	49 %	95 %

Four independent measurements moving together in the predicted direction, driven by a mechanism I can point at in the code: δ goes negative $\rightarrow$ the suspect sample's competitors are all masked $\rightarrow$ its loss contribution vanishes.

GAM-Softmax

Results and Demonstration of Final Result

What the numbers actually say

What the numbers actually say

1. The precondition failed.

AS - CE = -0.03 points over two seeds (-0.29 and +0.23: a coin flip). *Fourth testbed in a row.* The method I set out to generalise has, in my hands, no advantage over plain cross-entropy to generalise.

2. AS-Softmax memorises more than plain CE.

+13.6 vs +11.5 points over the floor, on both seeds (+7.8, +4.8). This contradicts the standard story — and my own reason for moving to noisy labels.

Mechanism: masking retires the easy negatives first — what a correctly-labelled example runs out of early. So an ever-larger share of the gradient goes to the still-confusable examples, which are exactly the mislabelled ones. Scalar masking concentrates effort onto the noise.

3. The sample axis does what it was designed to do.

(2 seed means)	CS	AS	MS	MS
misclassified ratio (pts)	32.5	33.9	32.4	32.1
misclass true label	61.2	60.1	60.9	57.3
prob vs true dir	5.80	5.80	2.67	3.23
non-target data masked	97%	95%	69%	65%

Four independent measurements moving together in the predicted direction, driven by a mechanism I can point at in the code: if gone negative → the suspect sample's competitors are all masked → its loss contribution vanishes.

Three readings. First, the precondition. Over two seeds AS-Softmax beats plain cross-entropy by minus 0.03 points — minus 0.29 on one seed, plus 0.23 on the other, which is a coin flip. That is the fourth testbed in a row where the method I set out to generalise has no reliable edge over the simplest possible baseline. You cannot generalise an advantage that is not there. Second, and this one surprised me: AS-Softmax memorises corrupted labels *more* than plain cross-entropy, on both seeds. Every margin-loss paper, and my own written rationale for moving to noisy labels, argues that masking should *prevent* memorisation. Measured directly, the opposite happens, and I think I know why — masking retires the easy competitors first, and easy competitors are what a correctly labelled example runs out of early, so as training goes on an ever-larger share of the gradient budget is spent on the hard examples, which are precisely the mislabelled ones. Scalar masking concentrates effort onto the noise. Third, the table: four independent measurements all move together for the sample-axis variant, and I can point at the line of code that causes it.

Verdict — stated as it is

	hypothesis	verdict
H0	AS > plain CE	NOT SUPPORTED — 3 of 4 testbeds show no advantage
H1	class-pair helps	NOT SUPPORTED — +0.12 over AS, inside a 1.0-pt noise band
H2	sample axis helps	SPLIT — peak accuracy not established; memorisation cut $\approx 4\times$
H3	the axes compose	UNTESTED — implemented, run cut for compute

finding	confidence
M4 cuts memorisation $\approx 4\times$ (-14.4 pts)	Confirmed (both seeds; $\approx 20\times$ the metric's spread)
AS-Softmax memorises <i>more</i> than CE	Confirmed (both seeds)
M4 gains at the final epoch (+2.77)	Confirmed (direction) (magnitude varies $2\times$)
H0 and H1 null	Confirmed (4 testbeds)
M4 gains at <i>peak</i> accuracy	Not established (one seed)
M4 is more seed-stable	Suggestive only ($n = 2$)
Calibration	No claim

The honest limit

M4's mean peak is +0.51 over CE — which would clear my +0.3 criterion. **I do not claim it.** Per seed it is -0.03 and $+1.04$: the entire mean comes from one seed where cross-entropy had a bad run. What *does* replicate is the *final-epoch* gain: +3.72 and +1.83, both positive, mean +2.77.

GAM-Softmax

Results and Demonstration of Final Result

Verdict — stated as it is

Verdict — stated as it is

hypothesis	verdict	finding	confidence
H0: All > place CE	NOT SUPPORTED — 3 of 4 verdicts show no advantage	M0 suits memorization in 4x4 (+0.44 pts)	Confirmed (both seeds) (+0.33) (no metric's spread)
H1: claim pair helps	NOT SUPPORTED — +0.12 more AS, inside a 1.64pt noise band	AS-Softmax mem- ories more than CE	Confirmed (both seeds)
H2: sample axis helps	SPLIT — peak ac- curacy not established	M0 gains at the final epoch (+0.77)	Confirmed (direction) (negative only, noise 2x)
H3: the axes compute	UNTESTED — sample memory not set for compute	M0 and M1 suit M0 gains at peak ac- curacy	Confirmed (+ verdicts) Not established (see note)
		M0 is more mem- orable	Suggestive only (x = 2)
		Calibration	No claim

The honest first

M0's mean peak is +0.51 over CE — which would clear any +0.3 criterion. I do not claim it. The seed it is -0.03 and +1.04; the entire mean comes from one seed where cross-entropy had a bad run. What does replicate is the end-of-training gain, positive on both seeds. On the right, every finding graded separately, because they are not equally solid.

The verdict table, left. My assumed precondition: not supported. The class-pair hypothesis, which was the original core of the project: not supported, and I showed earlier it is not a tuning artefact. The sample axis: split — it fails on the metric I pre-registered and succeeds on the mechanism metrics. The third hypothesis was implemented and unit-tested but I could not afford the compute to run it, and I would rather say that than quietly omit it. Then the box, which is the part I want to be judged on. The sample-axis variant's mean peak accuracy is plus 0.51 over cross-entropy, which would clear the plus 0.3 criterion I set in advance. I do not claim it. Per seed the gap is minus 0.03 and plus 1.04 — the entire mean comes from one seed where cross-entropy happened to have a bad run, and one seed of two is not a result. What does replicate is the end-of-training gain, positive on both seeds. On the right, every finding graded separately, because they are not equally solid.

Conclusion

In one sentence

Generalising AS-Softmax's scalar margin along the **class-pair** axis produced no measurable benefit on any of four testbeds; generalising it along the **sample** axis — by letting the margin go *negative*, which makes small-loss sample rejection a special case of a margin — did not reliably improve peak accuracy either, but **cut memorisation of corrupted labels roughly fourfold** and left the final-epoch model **2.8 points** better than cross-entropy under 40 % label noise.

What I would claim in writing

- the class-pair negative result, replicated over four testbeds and shown not to be a tuning artefact
- that AS-Softmax *increases* memorisation under label noise, contradicting the standard rationale for margin losses in that regime — measured directly, not inferred from accuracy
- the negative-margin reframing (small-loss rejection *is* a margin), with evidence that it works mechanically

What I would *not* claim

- that GAM-Softmax improves accuracy. It does not.
- any state-of-the-art noisy-label claim — I never ran the real baselines (co-teaching, DivideMix, GCE). M4 beats *cross-entropy and AS-Softmax at the final epoch*; that is a much weaker statement, and it is the only one the data supports.
- any calibration claim (the ECE gap is *underconfidence*; largely fixable by temperature scaling, which I did not run)

GAM-Softmax

Results and Demonstration of Final Result

Conclusion

The one-sentence conclusion is in the box, and it is written so that nothing in it exceeds the evidence. Then the two lists. On the left, what I would defend in writing: the negative result on the class-pair axis, replicated across four testbeds and shown not to be a tuning artefact; the finding that AS-Softmax increases memorisation under label noise, contradicting the standard rationale for margin losses in that regime — measured directly, not inferred from accuracy; and the negative-margin reframing (small loss rejection is a margin), with evidence that it works mechanistically.

Conclusion

In one sentence

Generalising AS-Softmax's scalar margin along the **class-pair** axis produced no measurable benefit on any of four testbeds; generalising it along the **sample** axis — by letting the margin go negative, which makes small-loss sample rejection a special case of a margin — did not reliably improve peak accuracy either, but cut memorisation of corrupted labels roughly fourfold and left the final speech model 2.8 points better than cross-entropy under 40% label noise.

What I would claim in writing

- the class-pair negative result, replicated over four testbeds and shown not to be a tuning artefact
- that AS-Softmax increases memorisation under label noise, contradicting the standard rationale for margin losses in that regime — measured directly, not inferred from accuracy
- the negative-margin reframing (small loss rejection is a margin), with evidence that it works mechanistically

What I would not claim

- that GAM-Softmax improves accuracy. It does not.
- any state-of-the-art noisy-label claim — I never ran the real baselines (no-tuning, GradRel, ECC), RM beats cross-entropy and AS-Softmax at the final speech task, is a much weaker statement, and it is the only one the data supports.
- any calibration claim (the ECC gap is underconfidence largely fixable by temperature scaling, which I did not test)

Limits, and what a follow-up should do differently

What would have to be true for this to be wrong

- 2 seeds, 1 dataset, 1 noise level, 1 architecture, 4 epochs. Two seeds is enough to kill a hypothesis and not enough to establish a small effect — which is exactly why the memorisation gap is claimed and the accuracy gap is not.
- β was never swept — the cleanest evidence for a mechanism is that the effect scales with its knob, and that is the run I could not afford
- symmetric noise is the easy case; real noise is instance-dependent, where “the model disagrees with this label” is a much weaker signal of corruption
- weak baselines by design — CE and AS-Softmax, not methods built for noisy labels

What a follow-up should do differently

1. **validate H0 in week 1**, not week 8. One week reproducing the baseline’s published gain would have redirected the whole project.
2. pick a regime where masking is *forced* to matter — 10k+ classes, or genuinely long-tailed data. Balanced text classification with a strong pretrained encoder is close to the worst case: plain CE is very hard to beat there.
3. instrument the mechanism from day 1. I read only accuracy for six weeks; it was flat everywhere and told me nothing about *why*.
4. keep the negative-margin framing, and test it against the real noisy-label baselines. It is one config line away.

Results and Demonstration of Final Result

Limits, and what a follow-up should do differently

The limits, stated before anyone has to ask. Two seeds, one dataset, one noise level — enough to kill a hypothesis, not enough to establish a small effect, which is precisely the distinction I drew two slides ago. Beta was never swept, and that is the honest hole: the cleanest evidence that a mechanism is what you think it is, is that the effect scales with its knob, and that is the run I could not afford. Symmetric noise is the easy case, and the baselines are weak by design. On the right, the four things I would do differently, and the first is the real lesson of the whole project: I assumed the baseline's advantage instead of spending one week checking it, and that assumption was the crux. Thank you — happy to take questions.

What would have to be true for this to be wrong

- 2 seeds, 1 dataset, 1 noise level, 1 architecture, 4 repeats. You need it enough to kill a hypothesis and not enough to establish a small effect — which is exactly why the communication gap is closed and the accuracy gap is not.
- If you never sweep — the observed evidence for a mechanism is that the effect scales with its knob, and that is the run I could not afford.
- Symmetric noise is the easy case; real noise is instance-dependent, where "the model disagrees with this label" is a much weaker signal of corruption.
- weak baselines by design — CE and MC-Softmax, not methods built for noisy labels.

What a follow-up should do differently

1. validate CE in *seed 1*, not *seed 0*. One week reproducing the baseline's published gap could have redirected the whole project.
2. pick a regime where making it *harder* to make — $10k+$ classes, or grossly long-tailed data. *Stated* test classification with a strong pretrained encoder is close to the worst case; please CE in very hard to beat there.
3. instrument the mechanism from day 1. I read only accuracy for six weeks; it was flat everywhere and told me nothing about why.
4. keep the negative-margin learning, and test it against the real noisy-label baselines. It is one curvy line away.

BACKUP SLIDES

For questions

Backup slides follow: per-seed numbers, the calibration caveat, and how to reproduce everything.

Backup — head-to-head, reported per seed

A two-seed mean can hide the fact that one seed did all the work, so here is every comparison split out.

vs. plain CE	peak (s42 / s43)	final (s42 / s43)	memorisation (s42 / s43)
as_softmax — CE	−0.29 / +0.23	+1.33 / −0.26	+7.8 / +4.6
gam_m3 — CE	+0.16 / +0.01	+0.67 / +1.38	+0.6 / +2.4
gam_m4 — CE	−0.03 / + 1.04	+3.72 / +1.83	−14.6 / −14.2

Read the M4 row across: the peak column disagrees with itself, the final and memorisation columns agree on both seeds. That difference is the entire basis for which claims I make and which I refuse to.

GAM-Softmax

└ Backup — head-to-head, reported per seed

This is the table that decides what I am willing to say. Look at the bottom row. The peak-accuracy column disagrees with itself — minus 0.03 on one seed, plus 1.04 on the other. The final-accuracy and memorisation columns agree on both seeds, and the memorisation effect is fourteen points in both cases. So I claim the second and third columns and refuse the first.

Backup — head-to-head, reported per seed

A two-seed mean can hide the fact that one seed did all the work, so here is every comparison split out.

vs. plain CE	peak (s42 / s43)	final (s42 / s43)	memorisation (s42 / s43)
a_suffmax - CE	-0.29 / +0.23	+1.33 / -0.26	+7.8 / +4.6
gam_m3 - CE	+0.16 / +0.01	+0.67 / +1.38	+0.6 / +2.4
gam_m4 - CE	-0.03 / +1.04	+3.72 / +1.63	-14.0 / -14.2

Read the M4 row across: the peak column disagrees with itself, the final and memorisation columns agree on both seeds. That difference is the entire basis for which claims I make and which I refuse to.

Backup — the calibration caveat

The margin variants have far *worse* ECE: 0.42–0.47 against plain cross-entropy's 0.19. Read naively that is a defeat. It is not what is happening.

- M4's mean predicted confidence is ≈ 0.25 while its accuracy is ≈ 0.67
- so the model is systematically **under**confident, not overconfident
- by construction: the margin objective *stops pushing* p_t upward once the gap is met, so the probabilities stay compressed

Underconfidence is a benign failure mode compared with cross-entropy's direction, and it is largely removable by temperature scaling — which I did not run. So: **no calibration claim in either direction.**

└ Backup — the calibration caveat

If someone asks about calibration: expected calibration error is much worse for the margin variants, 0.42 to 0.47 against 0.19. But look at the direction. Mean predicted confidence for the sample-axis variant is about 0.25 while its accuracy is about 0.67 — so it is badly *under* confident, not overconfident, and that is exactly what the objective is designed to do, because it stops pushing the true-class probability upward once the margin is met. Underconfidence is the benign direction and it is largely removable by temperature scaling, which I did not run. So I make no calibration claim either way.

The margin variants have far worse ECE: 0.42–0.47 against plain cross-entropy's 0.19. Read naively that is a defeat. It is not what is happening.

- M4's mean predicted confidence is ≈ 0.25 while its accuracy is ≈ 0.67
- so the model is systematically **underconfident**, not overconfident
- by construction: the margin objective stops pushing p_i upward once the gap is met, so the probabilities stay compressed

Underconfidence is a benign failure mode compared with cross-entropy's direction, and it is largely removable by temperature scaling — which I did not run. So: **no calibration claim in either direction.**

Backup — reproducing all of it

```
conda activate gam
```

```
python -m pytest tests/ -v                - 70 tests, ~3 s
python experiments/final_experiment.py      - E1, resumable (~6 h)
python experiments/final_experiment.py -only-block 1
```

- FINAL_REPORT.md — the full write-up
- HOW_WE_VARY_DELTA.md — the method in plain language
- runs/final_20ng/ — per-run JSON with full epoch histories, plus summary.md
- 9 losses in gam_softmax/losses/
- margins in gam_softmax/margins/
- one config = one experiment = one table row
- run 2026-07-28/31, commit b2ff750

GAM-Softmax

└ Backup — reproducing all of it

Everything is reproducible from the repository: the test suite runs in three seconds, and the final experiment is a single resumable command. Every number in this talk traces back to a JSON record under runs slash final-20ng, and the runs are stamped with the commit they came from.

```

unode activate gas

python -m pip install boto3 --index-url https://pypi.org/simple --trusted-host pypi.org
python experiments/final_experiment.py --index-url https://pypi.org/simple --trusted-host pypi.org
python experiments/final_experiment.py --only-final 1

```

- FINAL_REPORT.ad — the full write-up
- BOM_VARY_DELTA.ad — the method in plain language
- runs/final_20ng/ — per-run JSON with full epoch histories, plus summary.ad
- 9 losses in gas_softmax/losses/
- margins in gas_softmax/margins/
- one config :: one experiment :: one table row
- run 2026-07-28/31, commit b2ef750